

- **To pass:** submit a serious attempt to both problems.
- **Hand in:** upload on the course webpage. Solutions should be uploaded as a ZIP folder containing (1) the code that you have written and (2) a PDF 'report' (see below).
- **Presentation:** the second MinHash lecture will be used for the presentations. You will be asked to give a short presentation (5-10 minutes) of your solutions.
- **Collaboration:** feel free to discuss the problems together; but you should write up your solutions independently.

The assignment is separated into two tasks. In short:

Task 1 Estimate pairwise distances between a set of genomes using sketching and the Jaccard Index.

Task 2 Estimate an evolutionary tree for a set of genomes based on the distance estimates produced in Task 1.

Task 1

In this task, you should sample k -mers from the genomes using one¹ of the MinHash sketching techniques from Lecture 1. You should then use the sketches from the genomes to compute pairwise differences between the genomes. Let the distance between two genomes A and B be denoted as $d_{A,B}$. Then we define $d_{A,B} = 1 - J(A, B)$, where J is the Jaccard Index (lecture 1).

You do not have to implement your own hash function for the MinHash sketching techniques. Feel free to use the one(s) available in your programming language of choice. Your program should take k -mer size and sketching density as parameters, and the input data should be a file in fasta format (see the Data section below).

Task 1: Guiding questions

The questions below should be viewed as "sanity check" questions to debug your program and allow you to reflect on your solution.

1. Does the genomic distance estimates change with the density rate in sketching?
2. Does genomic distance change with the k -mer size used?

Note that some of the built-in hash functions (e.g., `hash()` in Python) change the initial value (seed) in the hash function based on the internal time stamp of the computer. Thus, the computed hash values for a given input will vary between runs. In such cases, you will note differences in the distance estimates.

¹You can, of course, implement several if you want.

Task 2

This task is more open ended than task 1. You should based on the distance estimates you computed in task 1 find a way to produce an *evolutionary tree* predicting the evolutionary history of organisms (Ex. in Fig 1). For this exercise, we will treat an evolutionary tree is a binary tree where leaf nodes represents the an organism and remaining nodes are common ancestors. You only need to care about the tree structure, i.e., not the lengths of the branches sometimes used to estimate the evolution time (molecular clock). Feel free to search the web for suitable methods based on your data (pairwise distances) produced in task 1.

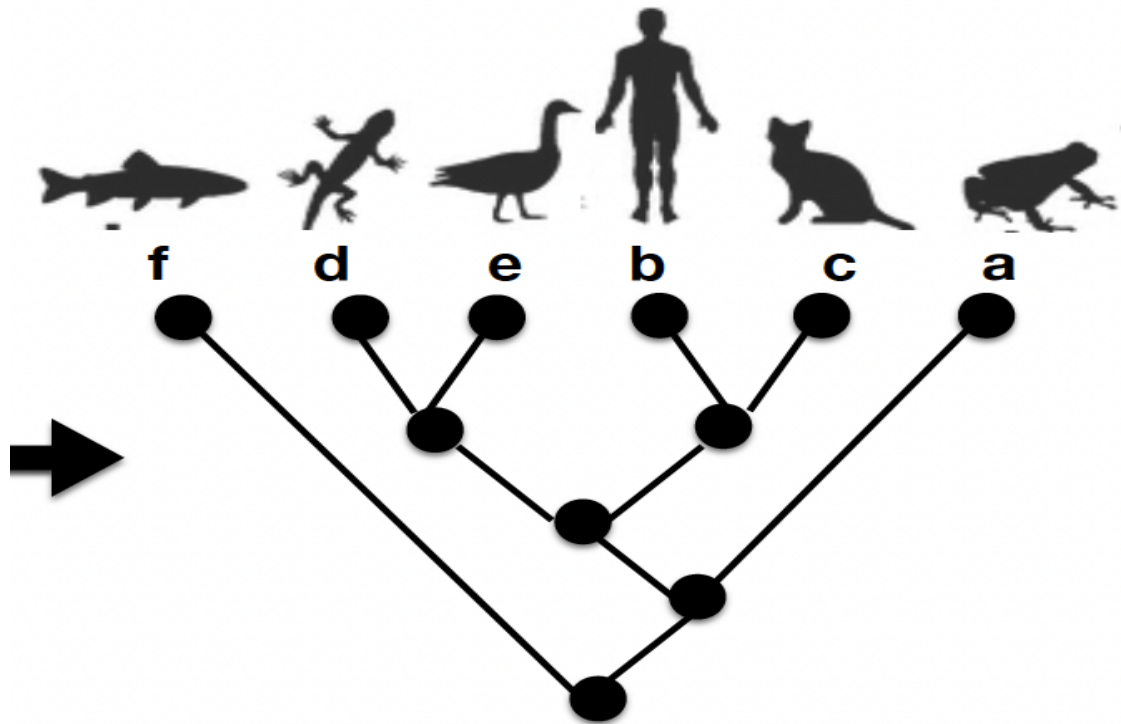


Figure 1: An evolutionary tree where leaf nodes represents (sequences of) an organism and internal nodes are common ancestors. From the tree structure, we can infer that humans and cats have a more recent common ancestor than, e.g., humans and frogs.

Guiding questions

1. Study the input data. Do genomes with small pairwise differences end up close to each other?
2. For the three test datasets where lineage is present in the accession, does your estimated tree agree with the lineage?

Data

There are four individual datasets to test your implementation on. There are three smaller simulated datasets labeled test1, test2, and test3 consisting of 3, 5, and 10 genomes, respectively. Then there is also a larger biological dataset consisting of 20 *E. coli* genomes to stress-test your implementation.

Input is formatted in the *Fasta format* (https://en.wikipedia.org/wiki/FASTA_format). Each genome in a Fasta file starts with one row containing the header genome identifier and description (denoted accession) of the genome and following lines containing the DNA of the genome. See the example below.

```
>genome1 Some info here
ACGGTCTAGCTACGATGCTCGATGCAGTCAT
ACATCTACTATCTATCT
>genome2 Other info here
CGGTATCGAGGCTGTATGCGTAGTCTGAGTC
CACAGCTGATCGTACTGAGATCTACGAGCCT
ACACAC
>genome3 Other info here
ACATGCAGTCATGATTTTTTCGGCGGAGCGT
CTGATCTTGCTCGATATCGATGATC
```

There is no requirement on runtime performance, but try to reason around the time complexity of your implementation. If your program does not finish instantly (substantially less than 1 second) on the three simulated datasets you should reflect over the implementation - likely the program will take a very long time on the biological dataset.

The report

Try to keep your report concise and to the point. Reflect over how you best present your solution. At minimum, your report should contain:

1. The programming language you used and instructions on how to run your program (and how to compile it if your implementation is in a compiled language).
2. Which methods you decided to use in tasks 1 and 2. Particularly, since task 2 is more open, you should describe carefully the approach you took.
3. What findings did you have and conclusions can you make? (Use the guiding questions as a direction)
4. If your program has issues, explain what they are and potential fixes.